

Многопоточное программирование в Go; IO в GO

Цель работы

Освоить основные механизмы конкурентности в Go: горутин, каналы (channel), пакет sync, паттерны «воркер пул» и «fan-in / fan-out», управление контекстом (context).

Задание

Необходимо реализовать консольное приложение для параллельной обработки набора текстовых файлов (логов, книг, статей) по следующему сценарию:

1. Чтение каталога с входными файлами.
2. Параллельная загрузка содержимого каждого файла.
3. Разбиение текста на слова (токенизация) и параллельная нормализация (приведение к нижнему регистру, удаление пунктуации).
4. Статистическая обработка: подсчёт количества каждого слова.
5. Агрегация результатов и вывод топ-N самых частотных слов.

Работа должна быть реализована с использованием конвейера (pipeline) из нескольких этапов, каждый этап выполняется в отдельных горутин.

Структура конвейера

Сканер файлов → Читатели файлов (воркеры) → Токенизаторы (воркеры) → Агрегатор → Вывод

1. Сканер файлов – главная горутина: проходит по каталогу, отправляет пути файлов в канал *jobs*.
2. Читатели файлов – N горутин: читают файл, отправляют его содержимое в канал *rawTexts*.
3. Токенизаторы – M горутин: получают текст, чистят его, разбивают на слова, отправляют слова в канал *words*.
4. Агрегатор – одна горутина: собирает слова в *map[string]int*, используя мьютекс или шардирование.
5. Вывод – после завершения всех этапов печатает топ слов.

Технические требования

Обязательно использовать:

- ⑩ *goroutine*
- ⑩ *channel* (буферизированный или небуферизированный)
- ⑩ *sync.WaitGroup* для ожидания завершения горутин
- ⑩ *sync.Mutex* или *sync.RWMutex* (или атомарные операции) при доступе к общей карте слов
- ⑩ *context.Context* – для корректного завершения конвейера по сигналу *SIGINT* (Ctrl+C) или при возникновении ошибки
- ⑩ Регулярные выражения или *strings.FieldsFunc* для токенизации.

Дополнительно:

- ⑩ Динамическое изменение количества воркеров (через контекст или сигнал).
- ⑩ Метрики (счётчик обработанных файлов, слов, время выполнения).
- ⑩ Шардирование карты слов вместо одного мьютекса (увеличение производительности).
- ⑩ Graceful shutdown – конвейер не теряет уже обработанные данные при остановке.

Порядок выполнения

1. Создайте каталог с тестовыми файлами (от 10 до 50 небольших текстовых файлов .txt).
2. Реализуйте функцию *scanDirectory*, которая возвращает канал строк (пути файлов).
3. Реализуйте функцию *runWorkers* для чтения файлов и токенизации (можно объединить, но лучше разделить).
4. Реализуйте агрегатор с защитой конкурентного доступа.
5. Используйте *context.WithCancel* для остановки по *os.Interrupt*.
6. Добавьте вывод статистики – время работы, количество обработанных файлов, уникальных слов.

Ожидаемый вывод программы

```
$ go run main.go -dir ./testdata -top 10 -workers 4
```

Обработано файлов: 12

Уникальных слов: 847

Время: 234ms

Топ-10 слов:

the (1204)

and (987)

to (754)

...